FlashAttention: IO-Aware Tiling.

The previous topic showed that materialising the $T \times T$ score matrix costs $O(T^2)$ memory. FlashAttention removes that cost *without changing the answer*: it tiles the computation and uses an *online softmax* with running statistics, so it never stores more than a block at a time — $O(T)$ memory, exact output, and a 2–4× speed-up from staying in fast on-chip memory. Every number below is computed in full.

THE RECURRENCE (online softmax)

$$m^{\text{new}} = \max(m,\, \tilde{m}), \quad \ell \leftarrow \ell\, e^{m - m^{\text{new}}} + \sum e^{s - m^{\text{new}}}, \quad O \leftarrow O\, e^{m - m^{\text{new}}} + \sum e^{s - m^{\text{new}}} v$$

Worked example. A length-6 score row split into two blocks of 3; we track (m, ℓ, O) block-by-block and verify it equals the one-shot softmax.

Topic 2 of 2 in this module · companion to the course page · v1.0

Read top to bottom: each step feeds the next.

Contents

1 The claim: exact attention in linear memory

FlashAttention computes the *same* function as Eq. (1) of Topic 1,

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V,$$

but reorganises the arithmetic so the full $T \times T$ score matrix is *never written to memory*. The compute stays $O(T^2 d)$ — you cannot avoid the T^2 pairs — yet the memory drops to linear:

$$\boxed{\text{compute } = O(T^2 d) \text{ (unchanged)}, \qquad \text{memory } = O(T) \text{ (was } O(T^2))} \tag{1}$$

The trick has two parts: (i) **tiling** — process keys/values in blocks so only one block is resident at a time; and (ii) the **online softmax** — a way to combine the partial results of each block into the exact final softmax, using two running scalars per query row: a running max m and a running normaliser ℓ.

A normal softmax needs the whole row before it can normalise (it must know the global maximum and the total sum). The online softmax instead keeps a *running* maximum and a *running* sum, and whenever a new, larger value appears it *rescales* everything seen so far by a single factor. It is the same trick as computing a running average without storing every sample — you keep a summary, not the data.

2 Streaming softmax: the recurrence

Consider one query row with scores $s_1, \dots, s_T$ (already divided by $\sqrt{d}$) and value vectors $v_1, \dots, v_T$. The exact output is

$$o = \sum_j p_j v_j, \qquad p_j = \frac{e^{s_j - m^\star}}{\sum_k e^{s_k - m^\star}}, \qquad m^\star = \max_k s_k,$$

where subtracting $m^\star$ is the standard numerically-stable softmax shift. We process the s_j in blocks. After some blocks we hold three running quantities:

$$m = \max_{\text{seen}} s_j, \qquad \ell = \sum_{\text{seen}} e^{s_j - m}, \qquad O = \sum_{\text{seen}} e^{s_j - m}\, v_j.$$

When a new block arrives with local max $\tilde{m}$, the global running max may grow. Let $m^{\text{new}} = \max(m, \tilde{m})$ and define the correction factor $\alpha = e^{\,m - m^{\text{new}}} \le 1$. Then update

$$m^{\text{new}} = \max(m,\ \tilde{m}), \qquad \alpha = e^{\,m - m^{\text{new}}} \tag{2a}$$
$$\ell^{\text{new}} = \alpha\, \ell + \textstyle\sum_{j \in \text{blk}} e^{\,s_j - m^{\text{new}}} \tag{2b}$$
$$O^{\text{new}} = \alpha\, O + \textstyle\sum_{j \in \text{blk}} e^{\,s_j - m^{\text{new}}}\, v_j \tag{2c}$$

After the last block, the answer is $o = O/\ell$. The factor α retro-actively re-bases the accumulated sum and output from the *old* reference max to the *new* one — no past data is needed, only the two summaries.

2.1 Proof that it equals the standard softmax

We show the running O/ℓ equals $\sum_j p_j v_j$. *Invariant:* after processing any prefix with running max m,

$$\ell = \sum_{j \in \text{prefix}} e^{s_j - m}, \qquad O = \sum_{j \in \text{prefix}} e^{s_j - m} v_j.$$

Base case: empty prefix, $\ell = 0$, $O = 0$, $m = -\infty$. *Inductive step:* assume the invariant for max m. A new block with max $\tilde{m}$ gives $m^{\text{new}} = \max(m, \tilde{m})$. For any already-counted index j, $e^{s_j - m^{\text{new}}} = e^{m - m^{\text{new}}} e^{s_j - m} = \alpha\, e^{s_j - m}$, so multiplying the old ℓ, O by α re-references them to m^{new}; adding the block's own $e^{s_j - m^{\text{new}}}$ terms extends the sums. Hence (2b)–(2c) preserve the invariant. At termination $m = m^\star$, so

$$\frac{O}{\ell} = \frac{\sum_j e^{s_j - m^\star} v_j}{\sum_j e^{s_j - m^\star}} = \sum_j p_j v_j,$$

which is the exact softmax-weighted output. ■

The only state carried between blocks is two scalars per query row, m and ℓ, plus the running output O of size d. That is $O(d)$ per query, $O(Td)$ total — independent of how many key blocks there are. The T^2 scores are produced, used, and discarded one block at a time; they never coexist in memory.

3 Worked example: length-6 row, two blocks of 3

Take one query row with scores and values

$$s = \begin{bmatrix} 1 & 3 & 2 & | & 5 & 4 & 0 \end{bmatrix}, \quad V = \left[\begin{array}{cc} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ \hline 2 & 0 \\ 0 & 2 \\ 1 & 2 \end{array}\right],$$

split into block 1 $= \{1, 3, 2\}$ and block 2 $= \{5, 4, 0\}$.

Reference (one-shot). With $m^\star = 5$, $\sum_j e^{s_j - 5} = 1.578055$, the weights are

$$p = \begin{bmatrix} 0.0116 & 0.0858 & 0.0316 & 0.6337 & 0.2331 & 0.0043 \end{bmatrix}, \quad o = pV = \begin{bmatrix} 1.314809 & 0.592094 \end{bmatrix}.$$

Block 1. Local max $\tilde{m} = 3$, so $m = 3$, $\alpha = e^{-\infty - 3} \to 0$ (nothing seen yet). The exponentials e^{s-3} are

$$e^{1-3} = 0.135335, \quad e^{3-3} = 1, \quad e^{2-3} = 0.367879,$$

$$\ell = 0.135335{+}1{+}0.367879 = 1.503215, \qquad O = 0.135335 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 1 \begin{bmatrix} 0 \\ 1 \end{bmatrix} + 0.367879 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.503215 \\ 1.367879 \end{bmatrix}.$$

Running output so far: $O/\ell = [\,0.334759,\ 0.909969\,]$ (the correct softmax of block 1 alone).

Block 2. Local max $\tilde{m} = 5$, so $m^{\text{new}} = \max(3, 5) = 5$ and the correction is

$$\alpha = e^{3-5} = e^{-2} = 0.135335.$$

The block exponentials e^{s-5} are $e^{5-5} = 1$, $e^{4-5} = 0.367879$, $e^{0-5} = 0.006738$, with block sum 1.374618 and block-value sum

$$1 \begin{bmatrix} 2 \\ 0 \end{bmatrix} + 0.367879 \begin{bmatrix} 0 \\ 2 \end{bmatrix} + 0.006738 \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2.006738 \\ 0.749227 \end{bmatrix}.$$

Now rescale the old accumulators by α and add the block:

$$\ell = 0.135335 \cdot 1.503215 + 1.374618 = 1.578055,$$

$$O = 0.135335 \begin{bmatrix} 0.503215 \\ 1.367879 \end{bmatrix} + \begin{bmatrix} 2.006738 \\ 0.749227 \end{bmatrix} = \begin{bmatrix} 2.074841 \\ 0.934357 \end{bmatrix}.$$

Finalise. $o = \dfrac{O}{\ell} = \dfrac{1}{1.578055} \begin{bmatrix} 2.074841 \\ 0.934357 \end{bmatrix} = \begin{bmatrix} 1.314809 \\ 0.592094 \end{bmatrix}.$

This is *exactly* the one-shot reference $[\,1.314809,\ 0.592094\,]$ — bit-for-bit, not an approximation. Watch the running output evolve as blocks stream in:

	m	ℓ	running O/ℓ
after blk 1	3	1.503215	$[\,0.334759,\ 0.909969\,]$
after blk 2	5	1.578055	$[\,1.314809,\ 0.592094\,]$

The arrival of the large score $s_4 = 5$ in block 2 forced m up from 3 to 5; the $\alpha = e^{-2}$ rescale correctly shrank block 1's contribution before the new block was added.

4 The real win: HBM ↔ SRAM IO

On a GPU, memory is a hierarchy: a large but *slow* HBM (high-bandwidth memory, tens of GB) and a tiny but *fast* on-chip SRAM (tens of KB to a few MB per streaming multiprocessor). Attention is *memory-bound*: the arithmetic is cheap relative to the cost of shuttling the $T \times T$ scores between HBM and the compute units.

A naive implementation writes the full S to HBM, reads it back for softmax, writes P, reads it again for PV — that is $\Theta(T^2)$ HBM accesses. FlashAttention instead loads blocks of Q, K, V into SRAM, computes a block of scores there, updates (m, ℓ, O) in place, and only ever writes the final $O \in \mathbb{R}^{T \times d}$ back. With SRAM of size M, the HBM traffic falls to

$$\text{naive: } \Theta(T^2) \quad \longrightarrow \quad \text{FlashAttention: } \Theta\Big(\tfrac{T^2 d^2}{M}\Big),$$

i.e. reduced by a factor $\sim M/d^2$. Fewer trips to slow memory is where the wall-clock 2–4× **speed-up** comes from — the FLOP count is unchanged, but the kernel stops waiting on memory.

FLOPs are not the bottleneck; *data movement* is. FlashAttention is faster not because it does less math but because it keeps the working set in fast SRAM and touches slow HBM as little as possible. Saving memory and saving time are the same act here.

5 Memory: standard vs FlashAttention

The activation memory for the score matrix (per head, fp16, 2 bytes/entry) versus FlashAttention, whose footprint is the running state $O(Td)$ with d=128:

T	Standard scores $2T^2$	FlashAttention $\sim 2Td$
512	512 KB	128 KB
2,048	8 MB	512 KB
8,192	128 MB	2 MB
32,768	2 GB	8 MB
131,072	32 GB	32 MB

At $T = 131{,}072$ the standard scores need 32 GB *per head*; FlashAttention needs 32 MB — a 1000× reduction that grows linearly with T. This is what makes 100k-token context feasible on a single GPU.

FlashAttention is **exact**, not an approximation. It is sometimes lumped together with sparse or low-rank methods (Modules 5.2–5.3) that *trade accuracy* for speed — it does no such thing. The output is identical to standard attention (up to floating-point reassociation); only the memory traffic and runtime change. Reaching for an approximation when FlashAttention already solves the memory problem exactly is a common and costly mistake.

6 Where this sits in the track

Method	Compute	Memory / Exact?
Standard attention (Topic 1)	$O(T^2d)$	$O(T^2)$ / exact
FlashAttention (here)	$O(T^2d)$	$O(T)$ / **exact**
Sparse patterns (Module 5.2)	$O(T\sqrt{T})$	$O(T\sqrt{T})$ / approximate
Low-rank / kernels (Module 5.3)	$O(T)$	$O(T)$ / approximate

FlashAttention is the default first move: it removes the memory wall *for free* (no quality cost). When even $O(T^2d)$ *compute* is too much, Modules 5.2–5.3 trade exactness for a sub-quadratic FLOP count.

A Reproducible (NumPy)

Inputs: $s = [1, 3, 2, 5, 4, 0]$, V as in the body, block size 3.

```
import numpy as np
s = np.array([1.,3,2,5,4,0])
V = np.array([[1,0],[0,1],[1,1],[2,0],[0,2],[1,2]], float)

# one-shot reference
w = np.exp(s - s.max()); w /= w.sum()
ref = w @ V                        # [1.314809, 0.592094]

# online softmax, blocks of 3
m, l, O = -np.inf, 0.0, np.zeros(2)
for idx in ([0,1,2], [3,4,5]):
    sb, Vb = s[idx], V[idx]
    m_new  = max(m, sb.max())
    alpha  = np.exp(m - m_new) if np.isfinite(m) else 0.0
    p      = np.exp(sb - m_new)
    l      = alpha*l + p.sum()
    O      = alpha*O + p @ Vb
    print(m_new, l, O/l)           # blk1: 3 1.503215 [0.334759 0.909969]
out = O / l                        # blk2: 5 1.578055 [1.314809 0.592094]
assert np.allclose(out, ref)       # exact, not approximate

# memory: standard 2*T^2 vs flash ~2*T*d (d=128), per head, fp16
for T in [512,2048,8192,32768,131072]:
    print(T, 2*T*T, 2*T*128)
```